ENTERPRISE APPLICATION DECELOPMENT

EAPD7111

ST10344257
Nkosinathi Mabena

1. Executive Summary and Introduction

As Chief Solutions Architect, this report outlines the strategic technical vision for the new Global Logistics Management System (GLMS). Currently "TechMove Logistics" reliance on a disjointed legacy system (spreadsheets, emails, manual calls) is causing severe data fragmentation, lost invoices and compliance failures regarding expired contracts.

To resolve these critical operational bottlenecks and support future growth, the GLMS will transition TechMove from a monolithic, outdated approach into a modern, service-oriented, and containerized enterprise environment. The scope of this project will deliver a centralized Contract Management Hub, automated Service Request Processing, and seamless Financial Integration.

To ensure the system aligns perfectly with TechMove's business goals, the architecture will be guided by the Zachman Framework. To guarantee robust and maintainable code, we will implement industry-standard GoF design patterns, specifically the Factory Method, Observer, and Strategy patterns. Finally, to address the Board's concerns regarding future growth, the GLMS is architected for high availability and immense scalability, utilizing horizontal scaling, load balancing, and advanced caching layers to easily accommodate sudden surges in global contracts.

2. Enterprise Architecture Framework Strategy (Theory)

To ensure the GLMS addresses TechMove's current operational bottlenecks, the architecture will be structured using the Zachman Framework. While the TOGAF Framework excels at guiding step-by-step process implementations, Zachman acts as a holistic, multi-perspective ontology.

This is critical for TechMove. Logistics is inherently an industry defined by its complexities: *what* is being shipped (Data), *how* it is processed (Function), and *where* it is moving globally (Network).

By mapping the GLMS against Zachman's specific interrogatives across different stakeholder perspectives, we create a master blueprint. This guarantees that as we transition away from the fragmented, monolithic legacy system, no element from high-level international freight contracts down to the

containerized Docker network topology is overlooked or misaligned with the Board's business goals.

Diagram 1.1 Enterprise Architecture Framework Strategy (Practical)

The matrix below maps the strategic business scope (Planner) and the logical system design (Designer) specifically for the TechMove logistics environment.

Perspective	What (Data)	How (Function)	Where (Network)
Planner (Scope Context) (Executive Board View)	Core Business Entities: • Global Freight Contracts • Client Profiles and Histories • Service Level Agreements (SLAs) • International Service Invoices	Core Business Processes: • Contract Lifecycle Management (Draft, Active, Expired) • Service Request Processing • International Freight Billing and Currency Conversion	Business Locations: • Global Logistics Hubs • Client Headquarters • International Shipping Routes • Regional Administrative Offices
Designer (System Model) (Architect View)	Logical Data Models: • ERD mapping 'Contract' to ServiceRequest entities (1-to-many). • JSON payload schemas for Financial API integrations. • Status tracking state definitions (Enums).	Application Architecture: • Service-Oriented Architecture logic (SOA). • Request validation algorithms (verifying active contract state prior to creation). • External API integration logic for automated currency data.	Distributed Infrastructure: • Cloud-hosted API Gateways. • Containerized Docker nodes hosting microservices • Load Balancers distributing global traffic

3. Design Pattern Selection and Implementation

Three specific Gang of Four (GoF) design patterns will be implemented; this is to ensure that the GLMS is highly scalable, testable, and loosely coupled. These patterns directly address the core complexities of contract creation, automated status tracking, and external financial integrations.

The Factory Method Pattern

Selection and Justification

The Factory Method is a creational design pattern that defines an interface for creating objects but let's subclasses decide which object to instantiate (geeksforgeeks 2026). TechMove handles various modes of logistics, requiring different types of service requests (e.g., AirFreightRequest, SeaFreightRequest). Hardcoding these instantiations directly within the client code would tightly couple the system to specific implementations, violating the Open/Closed Principle. The Factory Method solves this by decoupling the creation process. If TechMove expands to include RailFreight in the future, the core system remains untouched; we simply inject a new factory.

Logic and Structure

Architecturally, we will define an IServiceRequestFactory interface declaring a CreateRequest() method. Concrete creator classes (e.g., AirFreightFactory) will implement this interface to instantiate and return specific products that conform to an IServiceRequest interface. This structure heavily utilizes dependency injection, allowing the core Contract Management Hub to process generic service requests without ever needing to know their concrete types (Gemini 2026).

The Observer Pattern

Selection and Justification

The Observer Pattern is a behavioural design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing (refacroring.guru 2026). The GLMS requires that the system validate that service request are only raised against valid, active contracts and it must track status changes (Draft, Active, Expired).

Instead of the Service Request system constantly querying the database to check if a contract has expired, the contract proactively notifies the system the moment its status change. This prevents resource waste and ensures system components remain entirely decoupled.

Logic and Structure

Architecturally, the Contract class acts as the ‘Subject’. It maintains a private list of IObserver interfaces (subscribers). The ServiceRequestValidator and BillingIntegration classes will implement IObserver. When a contract reaches its expiry date, the Contract.UpdateStatus() method triggers NotifyObservers(), immediately pushing the state change to the validator to block any new logistics managers from raising requests against it (Gemini 2026).

The Strategy Pattern

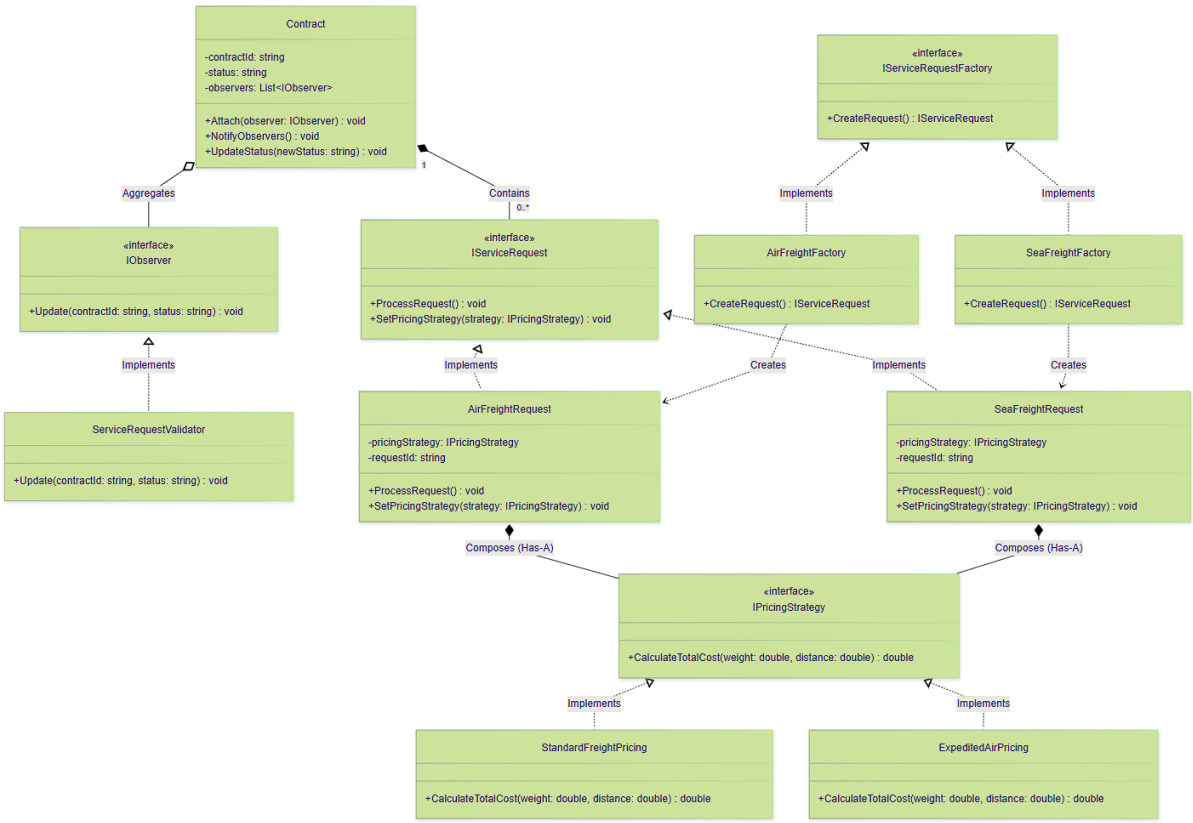
Selection and Justification

The Strategy Pattern is a behavioral pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable (refacroring.guru 2026). We know that TechMove handles diverse global contracts, each with its on Service level Agreement (SLA) and different billing structures for international freight. We must avoid hardcoding the cost calculation logic for every possible contract type using rigid if/else or switch statements as this would create unmaintainable code. Implementing the Strategy Pattern allows the GLMS to dynamically apply the correct pricing or routing logic at runtime without altering the core system.

Logic & Structure

We will define an IPricingStrategy interface containing a CalculateTotalCost() method. We will then build concrete strategy classes (e.g., StandardFreightPricing, ExpeditedAirPricing, HazardousMaterialPricing) that implement this interface. The ServiceRequest class acts as the Context; it holds a reference to the IPricingStrategy interface. When a request is processed, it delegates the calculation to the injected strategy, cleanly decoupling the complex financial rules from the request object itself (Gemini 2026).

Diagram 1.2 UML Modelling



4. Non-Functional Requirements and Scalability Strategy

To ensure the GLMS is an enterprise grade solution capable of supporting TechMove's aggressive growth, the system architecture needs to adhere to strict non-functional requirements and scalability protocols.

4.1 Non-Functional Requirements

Availability

As a global coordinator, TechMove operates across multiple time zones, meaning the GLMS must achieve a high availability of 99.9%. To guarantee this, the system's containerized Docker environment will be deployed across multiple availability zones. If a single node or regional server fails, automated failover mechanisms will instantly reroute traffic to healthy nodes, ensuring zero downtime for logistics managers raising critical service requests.

Interoperability

The system must seamlessly exchange data with external third-party systems, particularly for the Financial Integration module. By utilizing standardized RESTful APIs and JSON data payloads, the GLMS is fully interoperable. It can communicate securely and reliably with external international currency databases regardless of the underlying technology stack those external entities use.

4.2 Scalability Strategy (50,000 Contract Surge)

The Board has raised a concern regarding system stability in the event of a competitor acquisition, specifically a sudden influx of 50,000 new contracts. The GLMS is architected to handle this exact scenario through flexible infrastructure.

We will avoid relying on Vertical Scaling (adding more CPU or RAM to a single, expensive server), as this creates a monolithic single point of failure. Instead, the GLMS utilizes Horizontal Scaling. When the 50,000-contract surge occurs, the architecture will leverage the following mechanisms to handle the massive load:

Horizontal Scaling

The system will automatically spin up additional Docker container instances of the Contract Management microservice. This allows the system to process the massive data ingestion without slowing down the rest of the application.

Load Balancing

Cloud-based Load Balancers will act as traffic directors, preventing any single server from becoming a bottleneck during the surge. Incoming API requests and database write commands across the newly spun-up Docker nodes will be distributed evenly, ensuring maximum throughput.

Advanced Caching

Reading complex contract data repeatedly from the primary database is expensive. We will implement an in-memory caching layer (such as Redis) to prevent database timeouts during the surge. Frequently accessed data, such as active "Service Level Agreements" and daily exchange rates, will be cached. When thousands of Service Requests are being validated simultaneously, the

system will read from the lightning-fast cache rather than querying the database, drastically reducing latency.

REFERENCING LIST

GeeksforGeeks, 2023. Factory method design pattern in C++. [online] Available at: https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/ [Accessed 13 April 2026].

Refactoring.Guru, 2026. Observer. [online] Available at: https://refactoring.guru/design-patterns/observer [Accessed 13 April 2026].

Refactoring.Guru, 2026. Observer. [online] Available at: https://refactoring.guru/design-patterns/strategy [Accessed 13 April 2026].